How-To Library

Informatica

Configuring the Simba MongoDB JDBC Driver Options for MongoDB Connector

© Copyright Informatica LLC 2018, 2025. Informatica, the Informatica logo, and Informatica Cloud are trademarks or registered trademarks of Informatica LLC in the United States and many jurisdictions throughout the world. A current list of Informatica trademarks is available on the web at https:// www.informatica.com/trademarks.html.

Abstract

MongoDB Connector uses the Simba MongoDB JDBC driver to connect to MongoDB. This article provides detailed information about the MongoDB JDBC driver options that you can configure in a MongoDB connection in Data Integration.

Supported Versions

- Informatica Cloud® Data Integration MongoDB Connector

Table of Contents

Overview. 2
Configure the Simba MongoDB JDBC Driver Options in a MongoDB Connection. 2
Prerequisites. 2
Features. 2
Schema Editor. 3
Configure the Simba MongoDB JDBC Driver Properties. 5
Configure a MongoDB Connection. 8

Overview

You can use MongoDB Connector to connect to the MongoDB Atlas database from Data Integration. Configure a MongoDB connection and specify the MongoDB JDBC driver options in the **Additional Connection Properties** of the MongoDB connection.

Configure the Simba MongoDB JDBC Driver Options in a MongoDB Connection

Prerequisites

Before you configure the Simba MongoDB JDBC driver options in a MongoDB connection, you must complete the following prerequisites:

- Enable the MongoDB Connector license for your organization.
- Ensure that the driver supports MongoDB 3.4, 3.6, and 4.0.

Features

The Simba MongoDB JDBC Driver supports the double-buffering feature to improve driver performance during **SELECT** operations.

Double-Buffering

The Simba MongoDB JDBC Driver is capable of using double-buffering to improve driver performance during **SELECT** operations.

The impact of double-buffering depends on how the transfer speed of your network compares to the data processing speed of the driver. If the transfer speed is significantly higher, then enabling double-buffering allows the driver to make full use of the network's capabilities. Conversely, if the transfer speed is considerably lower, the additional processes involved in double-buffering might cause a decrease in performance.

To make optimal use of double-buffering, you need to set an appropriate buffer size. A buffer size that is too small might decrease performance, while a buffer size that is too large might diminish the performance improvements from double-buffering. If the transfer speed of the network is slow, the additional time spent processing a large buffer size might cause a decrease in performance.

For information about configuring double-buffering, see **EnableDoubleBuffer** and **BatchSize** properties described in "Configure the Simba MongoDB JDBC Driver Properties" on page 5.

Schema Editor

NoSQL data stores are able to store denormalized data, which does not follow the rules of data typing and structure that apply to traditional relational data. Denormalized data does not have a schema, and can include complex data types such as nested arrays or arrays of differently-typed elements. Because traditional ODBC and JDBC toolsets do not support these data structures, the data needs to be mapped to a relational form.

The Schema Editor is a Java application that enables you to create and modify schema definitions for NoSQL data stores. A schema definition serves as a database layer that maps NoSQL data to JDBC compatible formats so that you can use standard BI tools to work with the data.

Note: You can use the Schema Editor application only when the Secure Agent is packaged with JDK 8.

You can use the Schema Editor in conjunction with a Simba driver to create schema definitions. The driver generates the initial schema definition by sampling the data to detect its structure, and then assigning an appropriate data type to each column and generating virtual tables to represent complex data types. You can then use the options in the Schema Editor to manually modify the schema definition and make sure that your application can work with the data.

All Simba drivers that support schema definitions include the Schema Editor as part of the driver installation.

Using the Schema Editor for JDBC Connections

The Schema Editor application is packaged differently for ODBC and JDBC drivers. Depending on the platform that you are running and whether you are using a version of the application that is packaged with an ODBC driver or a JDBC driver, different system requirements apply, and different procedures and files are available for starting the application.

System Requirements

You must complete the system requirements before you can use the Schema Editor for MongoDB JDBC connections.

Complete the following system requirements:

- Windows
  The Schema Editor supports the following versions of Windows (32- and 64-bit editions are supported):
  - Windows Vista 7, 8, or 10
  - Windows Server 2008 or later
- Linux
  The Schema Editor supports the following Linux distributions (32- and 64-bit editions are supported):
  - Red Hat® Enterprise Linux® (RHEL) 5, 6, or 7
  - CentOS 5, 6, or 7

- SUSE Linux Enterprise Server (SLES) 11 or 12
- Debian 7 or 8
- Ubuntu 12.04, 14.04, or 16.04

To use the Schema Editor on Linux, you must have GTK+ 2.18, 2.20, or 2.24 installed.

- macOS
  The Schema Editor supports macOS version 10.9, 10.10, and 10.11.

Start the Schema Editor for JDBC Connections

Use the `SchemaEditor.jar` file included in your JDBC driver package to start the Schema Editor. The application must be run using JRE 8u66 or later.

Perform the following steps to start the Schema Editor:

1. Open a Command Prompt or Terminal window and then navigate to the root directory of the driver.
2. Run one of the following commands, where `[JRE_Path]` is the full path to the `bin` subfolder in the JRE directory.
   - On Windows:
     `[JRE_Path]\java -jar SchemaEditor.jar`
   - Or, on Linux or macOS:
     `[JRE_Path]/java -jar SchemaEditor.jar`

     Note: If your machine uses JRE 8u66 or later by default, then it is not necessary to include `[JRE_Path]` in the command.

   The Schema Editor opens on the start page. From there, you can choose to create a new schema definition by sampling data, or modify an existing schema definition stored in a JSON file or a NoSQL data store.

Start the Schema Editor from the Secure Agent location

You can start the Schema Editor from the following location:

```
<Secure Agent installation directory>\downloads\package-mongodbadapter.xxx\package\mongodb
\thirdparty\informatica.mongodb\
```

Execute the following command to start the Schema Editor:

- For Windows: java -cp ".;*;" com.informatica.mongodb.schemaeditor.SchemaEditor
- For Linux: java -cp ".:*:" com.informatica.mongodb.schemaeditor.SchemaEditor

Configure the Simba MongoDB JDBC Driver Properties

When you create a MongoDB connection, configure the **Additional Connection Properties** to specify the Simba MongoDB JDBC driver properties.

The following table describes the Simba MongoDB JDBC driver properties that you can configure in a MongoDB connection:

Property	Description
AuthMechanism	The authentication mechanism that the driver uses to connect to the MongoDB server. You can specify one of the following values: - None. The driver does not authenticate the MongoDB connection. - SCRAM-SHA-1. The driver authenticates the MongoDB connection using the SCRAM-SHA-1 protocol, which is the default authentication protocol used by MongoDB. - GSSAPI. The driver authenticates the MongoDB connection using the Kerberos protocol. - PLAIN. The driver authenticates the MongoDB connection using the LDAP protocol.
AuthSource	The name of the MongoDB database that you want to authenticate.
BatchSize	The maximum number of documents that a query returns at a time. Default is 4096.
CatalogSchemaSwap	Specifies whether the driver treats MongoDB databases as schemas or catalogs. If you specify **true**, the driver treats MongoDB databases as schemas. If you specify **false**, the driver treats MongoDB databases as catalogs.
connectTimeoutMS	The number of milliseconds to wait during connection attempts before timing out. If connecting to the server takes longer than the specified amount of time, then the driver stops the connection attempt.
DefaultStringColumnLength	The maximum number of characters that a STRING column can contain. You can specify a maximum value of 2147483647. Default is 255.
DmlBatchSize	The maximum number of documents that the driver can handle at one time during a write operation. Default is 500.
EnableDoubleBuffer	Specifies whether the driver retrieves the data using double-buffering or single-buffering. If you specify **true**, the driver uses double-buffering to retrieve the data. If you specify **false**, the driver uses single-buffering to retrieve the data.
EnableMixedTypeFilter	Specifies whether the driver uses mixed type filtering, which allows a value to pass the filter even if it is not the same data type as the specified filter term. If you specify **true**, the driver retrieves all values that match the specified filter term, even if the values are stored as different data types than the filter term. If you specify **false**, the driver only retrieves values that match the data type of the specified filter term. Default is true.

Property	Description
EnableTransaction	Specifies whether the driver supports connections where auto-commit mode is disabled. If you specify **true**, the driver supports connections where auto-commit mode is disabled. If you specify **false**, the driver does not support connections where auto-commit mode is disabled. Default is false.
gssapiServiceName	The Kerberos service principal name of the MongoDB server. Default is **mongodb**.
JSONColumnSize	Specifies the default column length for JSON fields. Default length is **1023**.
LoadMetadataTable	Specifies where the Secure Agent looks for the schema definition. If you specify **true**, the Secure Agent loads the schema definition from the MongoDB database. If you specify **false**, the Secure Agent loads the schema definition from the JSON file specified in the **LocalMetadataFile** key.
LocalMetadataFile	Specifies the complete path and name of JSON file containing the schema definition that the Secure Agent uses to connect to MongoDB.
LogLevel	Species whether the Secure Agent logs error messages in the session log. You can specify one of the following values: - 0: Disable all logging. - 1: Enable logging on the FATAL level, which logs very severe error events that will lead the driver to abort. - 2: Enable logging on the ERROR level, which logs error events that might still allow the driver to continue running. - 3: Enable logging on the WARNING level, which logs events that might result in an error if action is not taken. - 4: Enable logging on the INFO level, which logs general information that describes the progress of the driver. - 5: Enable logging on the DEBUG level, which logs detailed information that is useful for debugging the driver. - 6: Enable logging on the TRACE level, which logs all driver activity. When logging is enabled, the driver produces the following log files in the location specified in the **LogPath** property: - A `MDBJDriver_driver.log` file that logs driver activity that is not specific to a connection. - A `MongoDBJDBC_connection_[Number].log` file for each connection made to the database, where [Number] is a number that distinguishes each log file from the others. This file logs driver activity that is specific to the connection.
LogPath	The complete path to the folder where the driver saves log files when logging is enabled.
NoCursorTimeout	Specifies whether the driver allows active cursors on the data source server to expire. If you specify **true**, the driver prevents the data source server from timing out idle cursors. If you specify **false**,The data source server times out idle cursors when the inactivity threshold specified on the server transpires **Note:** When this property is set to True, if the driver quits or loses the connection to the server unexpectedly, there is a risk of the cursor remaining open on the server indefinitely. You can adjust the threshold for idle cursor timeouts on the MongoDB server.

Property	Description
ReplicaSet	If you specify **true**, The driver reports the JSON column. Name of the replica set of the database.
SamplingLimit	The maximum number of records that the Secure Agent can sample to generate the schema definition. To sample every record in the database, set this option to 0. Default is 100.
SamplingStepSize	The interval at which the Secure Agent samples records when scanning through the database to generate a temporary schema definition. For example, if you set the sampling interval to 2, the Secure Agent samples every second record in the database. Default is 1.
SamplingStrategy	Specifies how the Secure Agent samples data when generating a temporary schema definition. You can specify one of the following values: - Forward: The Secure Agent samples data starting from the first record in the database, then samples the next record, and so on. - Backwards: The Secure Agent samples data starting from the last record in the database, then samples the preceding record, and so on. Default is Forward.
SocketTimeout	Specifies the amount of time, in milliseconds, before an attempt to send or receive on a socket time out. When this property is set to `-1`, the driver does not specify the amount of time before a socket connection times out.
SSL	Specifies whether the Secure Agent establishes a secure connection to the MongoDB server using SSL. If you specify **true**, the Secure Agent establishes an encrypted connection to the MongoDB server. If you specify **false**, the Secure Agent establishes an unencrypted connection to the MongoDB server. Not Applicable for MongoDB Connector.
sslTrustStore	Applicable only if you enable SSL. The path and file name of the truststore file. Not Applicable for MongoDB Connector.
sslTrustStorePwd	The password for the truststore file. Not Applicable for MongoDB Connector.
UseJSONColumn	Specifies whether the driver reports a special column named DocumentAsJson that retrieves or stores whole documents as JSON-formatted strings. If you specify **true**, The driver reports the JSON column. If you specify **false**, The driver does not report the JSON column. **Note:** Only enable this option as needed. The process of converting documents into JSON strings decreases the performance of the driver.

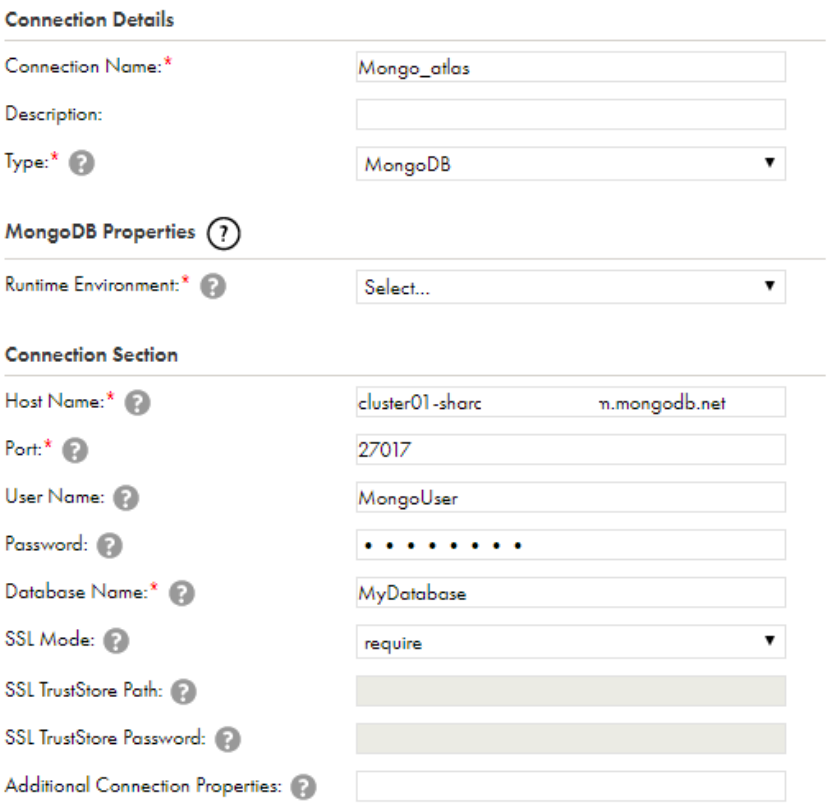
Property	Description
WriteConcern	Specifies the total number of primary and secondary servers that the driver uses to report a successful write operation. Default is 1. **Note:** If you use a value that is greater than **1**, you must specify an appropriate value for the **WriteConcernTimeout** property.
WriteConcernJournaled	Specifies whether the driver requires the data from a write operation to be committed to the journal before the write operation can be acknowledged. If you specify **1**, the driver requires the data from to be committed to the journal. If you specify **0**, the driver does not require the data to be committed to the journal. Default is 1.
WriteConcernTimeout	The maximum number of seconds that the driver waits for a secondary server to acknowledge a write operation before reporting that the operation has failed. By default, or when this property is set to 0, the driver does not time out. Instead, the driver waits until all secondary servers acknowledge the write operation, and then reports that the operation has succeeded. This option is applicable only when **WriteConcern** is enabled.

Configure a MongoDB Connection

You must configure a MongoDB connection to connect to a MongoDB database. You can configure a connection on the **Connections** page in Administrator or in a wizard when you configure a mapping.

1. Perform either of the following steps:
   - In Administrator, select **Connections**.
   - In Data Integration, open a Source or Target transformation in a mapping.
2. Click **New Connection**.

The **Connection Details** dialogue appears:



3. Configure the following connection details:

Property	Description
Connection Name	The name of the connection. The name is not case sensitive and must be unique within the domain. You can change this property after you create the connection. The name cannot exceed 128 characters, contain spaces, or contain the following special characters: ~ ` ! $ % ^ & * ( ) - + = { [}] | \ : ; " ' < , > . ? /
Description	Optional. The description of the connection. The description cannot exceed 4,000 characters.
Type	The connection type. Select **MongoDB**.

4. Configure the following connection properties:

Property	Description
Runtime Environment	Name of the runtime environment where you want to run the tasks.
Host Name	Host name or IP address of the MongoDB server.
Port	MongoDB server port number. Default is 9042.
User Name	User name to access the MongoDB server.
Password	Password corresponding to the user name to access the MongoDB server.
Database Name	Name of the MongoDB database to connect to.
Additional Connection Properties	Enter one or more JDBC connection parameters in the following format: `<param1>=<value>&<param2>=<value>&<param3>=<value>` You must provide the JDBC parameters as ampersand-separated key-value pairs. MongoDB Connector supports the following JDBC connection parameters: - AuthSource - BatchSize - connectTimeoutMS - DefaultStringColumnLength - DmlBatchSize - EnableDoubleBuffer - EnableTransaction - LogLevel - LogPath - SamplingLimit - SamplingStepSize - SamplingStrategy For example, DefaultStringColumnLength=512&DmlBatchSize=1000& EnableDoubleBuffer=false&EnableTransaction=true& SamplingLimit=200&SamplingStepSize=2&SamplingStrategy=Backwards **Note:** If you specify the host name, port number, user name, and password of the MongoDB server in the **Additional Connection Properties**, the values specified in the **Additional Connection Properties** takes precedence.
SSL Mode	Specifies whether the Secure Agent establishes a secure connection to the MongoDB server. You can select one of the following options: - disabled. The Secure Agent establishes an unencrypted connection to the MongoDB server. - require. The Secure Agent establishes an encrypted connection to the MongoDB server without authentication. - one-way. The Secure Agent establishes an encrypted connection to the MongoDB server using truststore file and truststore password. Default is disabled. Not Applicable for MongoDB Connector.

Property	Description
SSL TrustStore Path	The path and file name of the truststore file. Applicable only if you select **one-way** in the **SSL Mode** property. Not Applicable for MongoDB Connector.
SSL TrustStore Password	The password for the truststore file. Applicable only if you select **one-way** in the **SSL Mode** property. Not Applicable for MongoDB Connector.

5. To test the MongoDB connection, click **Test Connection**.

   If a database connection fails, contact the database administrator.

6. Click **Save** to save the MongoDB connection.

Authors

Ripa Bhagawati

Anupam Nayak

Acknowledgements

The author would like to acknowledge Tanjot Singh Uppal and Akshatha Kamath Karkala, for their technical assistance.